

301AA - Advanced Programming

Lecturer: **Andrea Corradini**

andrea@di.unipi.it

<http://pages.di.unipi.it/corradini/>

AP-25: Lambdas and Streams in Java 8

Java 8: language extensions

Java 8 is the biggest change to Java since the inception of the language. Main new features:

- **Lambda expressions** (Anonymous functions)

- Method references
- Default methods in interfaces
- Improved type inference

A big challenge was to introduce lambdas without requiring recompilation of existing binaries

- **The Stream API**

- Conceptually, working with streams in Java is very much like working with **lists** in Haskell using higher order combinators (not recursion)

Benefits of Lambdas in Java 8

- Enabling functional programming
 - Being able to pass behaviors as well as data to functions
 - Introduction of **lazy evaluation** with stream processing
- Writing cleaner and more compact code
- Facilitating parallel programming
- Developing more generic, flexible and reusable APIs

Passing behaviours in Java pre-8 vs. Java 8

```
JButton button = new JButton("Click Me!");

// pre Java 8
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent evt) {
        System.out.println("Handled by anonymous class listener");
    }
});

// Java 8
button.addActionListener(
    e -> System.out.println("Handled by Lambda listener"));
```

Passing behaviour...

```
public class ThreadTest {// using functional interface Runnable
    public static void main(String[] args) {
        Runnable r1 = new Runnable() { // anonymous inner class
            @Override
            public void run() {
                System.out.println("Old Java Way");
            }
        };

        new Thread(r1).start();
        // using lambda expression
        Runnable r2 = () -> { System.out.println("New Java Way"); };

        new Thread(r2).start();
    }
}
```

```
// constructor of class Thread
```

```
public Thread(Runnable target)
```

Lambda expression syntax:

Print a list of integers with a lambda

```
List<Integer> intSeq = Arrays.asList(1,2,3);  
  
intSeq.forEach(x -> System.out.println(x));  
  
x -> System.out.println(x)
```

is a **lambda expression** that defines an **anonymous function (method)** with one parameter named **x** of type Integer

```
// equivalent syntax  
intSeq.forEach((Integer x) -> System.out.println(x));  
  
intSeq.forEach(x -> {System.out.println(x);});  
  
intSeq.forEach(System.out::println); //method reference
```

- Type of parameter inferred by the compiler if missing

Multiline lambda, local variables, no new scope

```
List<Integer> intSeq = Arrays.asList(1,2,3);  
// multiline: curly brackets necessary  
// local variable declaration  
intSeq.forEach(x -> {  
    int y = x + 2;  
    System.out.println(y);  
});  
  
// no new scope!!!  
int x = 0, y = 0;  
intSeq.forEach(x -> {           //error: x already defined  
    double y = 2.0;           //error: y already defined  
    System.out.println(x + y);  
});
```

Assigning, passing as argument and returning lambdas

```
// lambda expressions can be assigned and returned
import java.util.*;
import java.util.function.*;
class Lambdas {
    public static void main(String... args){
        Arrays.asList(args).forEach(print());
    }

    public static Consumer<String> print(){
        Consumer<String> cons = (y -> System.out.println(y));
        return cons;
    }
}
```

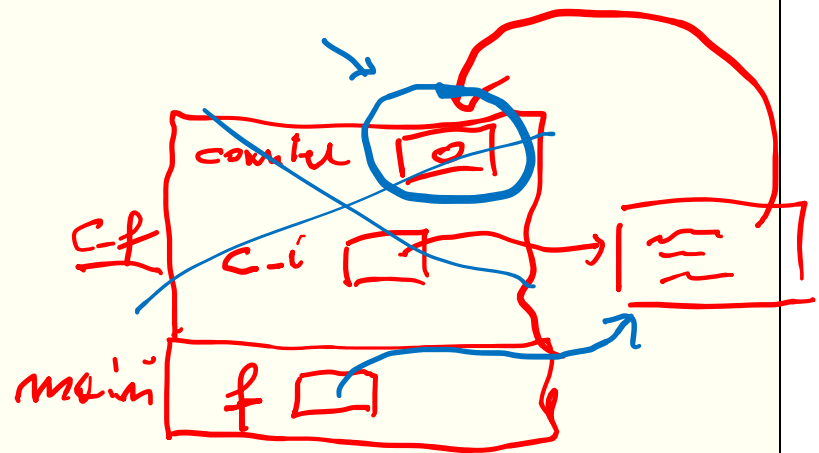
Recap: Closures

- **Closures** needed in general when functions can be passed to or returned by functions

```
def counter_factory(): c-f  
    counter = 0  
    def counter_increaser(): c-i  
        nonlocal counter  
        counter = counter + 1  
        return counter  
    return counter_increaser
```

```
>>> f = counter_factory()  
>>> f()  
1  
>>> f()  
2  
>>> f.__closure__  
(<cell at 0x1033ace88: int object at 0x10096dce0>,)
```

Python



Local and Static Variable Capture

- Local variables used inside the body of a lambda must be **final** or **effectively final**

```
public class LVCEExample {    // local variable capture
    public static void main(String[] args) {
        List<Integer> intSeq = Arrays.asList(1,2,3);

        int var = 10;          // must be [effectively] final
        intSeq.forEach(x -> System.out.println(x + var));
        // var = 3; // uncommenting this line it does not compile
    }
}
```

This is a fundamental design choice, as it makes closures not necessary

```
public class SVCEExample {    // static variable capture
    private static int var = 10;
    public static void main(String[] args) {
        List<Integer> intSeq = Arrays.asList(1,2,3);

        intSeq.forEach(x -> System.out.println(x + var));
        var = 3;              // it compiles
    }
}
```

Implementation of Java 8 Lambdas

- The Java 8 compiler conceptually first converts a lambda expression into a function, compiling its code
- Then it generates code to call the compiled function where needed
- For example, `x -> System.out.println(x)` could be converted into a generated static function

```
public static void genName(Integer x) {  
    System.out.println(x);  
}
```
- But what type should be generated for this function? How should it be called? What class should it go in?

Functional Interfaces

- Design decision: Java 8 lambdas are instances of *functional interfaces*.
- A **functional interface** is a Java interface with **exactly one abstract method**. E.g.,

```
public interface Comparator<T> { //java.util
    int compare(T o1, T o2);
}

public interface Runnable { //java.lang
    void run();
}

public interface Consumer<T>{ //java.util.function
    void accept(T t)
}

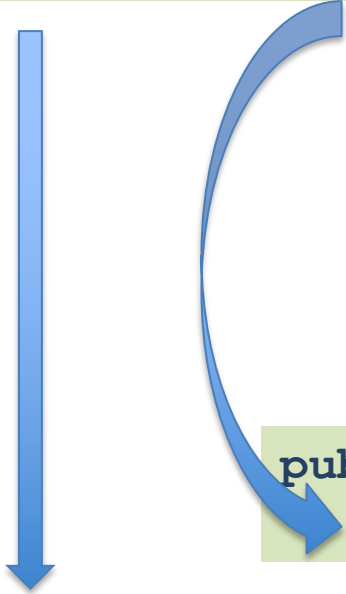
public interface Callable<V> { //java.util.concurrent
    V call() throws Exception;
}
```

Functional interfaces and lambdas

- Functional Interfaces can be used as *target type* of lambda expressions, i.e.
 - As type of variable to which the lambda is assigned
 - As type of formal parameter to which the lambda is passed
- The compiler uses type inference **based on target type**
- Arguments and result types of the lambda must match those of the unique abstract method of the functional interface
- The lambda is invoked by calling the only abstract method of the functional interface
- Lambdas can be interpreted as **instances** of **anonymous inner classes** implementing the functional interface

Expanding a lambda

```
List<Integer> intSeq = Arrays.asList(1,2,3);  
intSeq.forEach(x -> System.out.println(x));
```



```
// List<T> extends Iterable<T>  
interface Iterable<T>{           //java.lang  
    default void forEach(Consumer<? super T> action)  
        for (T t : this)  
            action.accept(t);
```

```
public interface Consumer<T>{ //java.util.function  
    void accept(T t); }        //functional interface
```

```
List<Integer> intSeq = Arrays.asList(1,2,3);  
for (Integer t:intSeq)  
    System.out.println(t);
```

An example: From inner classes pre Java 8...

```
public class Calculator1 { // Pre Java 8
    interface IntegerMath { // (inner) functional interface
        int operation(int a, int b);
    }
    public int operateBinary(int a, int b, IntegerMath op) {
        return op.operation(a, b);
    } // parameter type is functional interface

    // inner class implementing the interface
    static class IntMath$Add implements IntegerMath{
        public int operation(int a, int b){
            return a + b;
        }
    }

    public static void main(String... args) {
        Calculator1 myApp = new Calculator1();
        System.out.println("40 + 2 = " +
            myApp.operateBinary(40, 2, new IntMath$Add()));
        // anonymous inner class implementing the interface
        IntegerMath subtraction = new IntegerMath(){
            public int operation(int a, int b){
                return a - b;
            }
        };

        System.out.println("20 - 10 = " +
            myApp.operateBinary(20, 10, subtraction));    }}
```

... to lambda expressions

```
public class Calculator {  
  
    interface IntegerMath { // (inner) functional interface  
        int operation(int a, int b);  
    }  
  
    public int operateBinary(int a, int b, IntegerMath op) {  
        return op.operation(a, b);  
    } // parameter type is functional interface  
  
    public static void main(String... args) {  
        Calculator myApp = new Calculator();  
        // lambda assigned to functional interface variables  
        IntegerMath addition = (a, b) -> a + b;  
        System.out.println("40 + 2 = " +  
            myApp.operateBinary(40, 2, addition));  
        // lambda passed to functional interface formal parameter  
        System.out.println("20 - 10 = " +  
            myApp.operateBinary(20, 10, (a, b) -> a - b));  
    }  
}
```

From Lambdas to Bytecode

- Lambdas can, in principle, be compiled as instances of anonymous inner classes

```
List<Integer> intSeq = Arrays.asList(1,2,3);  
intSeq.forEach(x -> System.out.println(x));
```

```
List<Integer> intSeq = Arrays.asList(1,2,3);  
intSeq.forEach(new Consumer<Integer>(){  
    public void accept(Integer x){  
        System.out.println(x);  
    }  
});
```

- Neither JLS 8 nor JVM 8 prescribe a specific compilation strategy for lambdas:
- Current compilers use the **invokedynamic** instruction of the JVM. It allows to defer to runtime the computation of a call-site.

New Functional Interfaces in package java.util.function

```
public interface Consumer<T> {           //java.util.function
    void accept(T t);
}
public interface Supplier<T> {           //java.util.function
    T get();
}
public interface Predicate<T> {           //java.util.function
    boolean test(T t);
}
public interface Function<T,R> {           //java.util.function
    R apply(T t);
}
```

Other examples of lambdas

```
List<Integer> intSeq = new ArrayList<>(Arrays.asList(1,2,3));

// sort list in descending order using Comparator<Integer>
intSeq.sort((x,z) -> z - x); // lambda with two arguments
intSeq.forEach(System.out::println);

// remove odd numbers using a Predicate<Integer>
intSeq.removeIf(x -> x%2 == 1);
intSeq.forEach(System.out::println); // prints only '2'
```

But, where do the **sort**, **removeIf** and **forEach** methods come from?

```
// default method of Interface List<E>
default void sort(Comparator<? super E> c)
// default method of Interface Collection<E>
default boolean removeIf(Predicate<? super E> filter)
// default method of Interface Iterable<T>
default void forEach(Consumer<? super T> action)
```

Default Methods

Problem: Adding new abstract methods to an interface breaks existing implementations of the interface

Java 8 allows interface to include

- Abstract (instance) methods, as usual
- **Static methods**
- **Default methods**, defined in terms of other possibly abstract methods

Java 8 uses lambda expressions and default methods in conjunction with the Java collections framework to achieve ***backward compatibility*** with existing published interfaces

Method References

- Method references can be used to pass an existing function in places **where a lambda is expected**
- The signature of the referenced method needs to match the signature of the **functional interface method**

Method Reference Type	Syntax	Example
static	ClassName::StaticMethodName	String::valueOf
constructor	ClassName::new	ArrayList::new
specific object instance	objectReference::MethodName	x::toString
arbitrary object of a given type	ClassName::InstanceMethodName	Object::toString

Streams in Java 8

The **java.util.stream** package provides utilities to support functional-style operations on streams of values. **Streams** differ from **collections** in several ways:

- **No storage**. A stream is not a data structure that stores elements; instead, it conveys elements from a **source** (a data structure, an array, a generator function, an I/O channel,...) through a **pipeline** of computational operations.
- **Functional in nature**. An operation on a stream produces a result, but does not modify its source.

Streams in Java 8 (cont'd)

- **Laziness-seeking**. Many stream operations, can be implemented lazily, exposing opportunities for optimization. Stream operations are divided into **intermediate** (stream-producing) operations and **terminal** (value- or side-effect-producing) operations. ***Intermediate operations are always lazy.***
- **Possibly unbounded**. Collections have a finite size, streams need not. Short-circuiting operations such as ***limit(n)*** or ***findFirst()*** can allow computations on infinite streams to complete in finite time.
- **Consumable**. The elements of a stream are only visited once during the life of a stream. Like an ***Iterator***, a new stream must be generated to revisit the same elements of the source.

Pipelines

- A typical pipeline contains
 - A **source**, producing (by need) the elements of the stream
 - Zero or more **intermediate operations**, producing streams
 - A **terminal operation**, producing side-effects or non-stream values
- Example of typical pattern: filter / map / reduce:
“Compute the average age of males in a collection of people”

```
double average = listing // collection of Person
    .stream()             // stream wrapper over a collection
    .filter(p -> p.getGender() == Person.Sex.MALE) // filter
    .mapToInt(Person::getAge) // extracts stream of ages
    .average()              // computes average (reduce/fold)
    .getAsDouble();         // extracts result from OptionalDouble
```

“Compute the average age of males in a collection of people”

```
double import java.util.NoSuchElementException;

// listing: Collection<Person>
double average;
int sum = 0;
int count = 0;

for (Person p : listing) {
    if (p.getGender() == Person.Sex.MALE) {
        sum += p.getAge(); // mapToInt(Person::getAge)
        count++;           // used for the average
    }
}

if (count == 0) {
    throw new NoSuchElementException("No value present"); //
    like OptionalDouble.getAsDouble()
}

average = (double) sum / (double) count;
```

Anatomy of the Stream Pipeline

- A Stream is processed through a **pipeline** of operations
- A Stream starts with a **source**
- **Intermediate methods** are performed on the Stream elements. These methods produce Streams and are not processed until the **terminal method** is called.
- The Stream is considered **consumed** when a terminal operation is invoked. No other operation can be performed on the Stream elements afterwards
- Some intermediate or terminal methods can be **short-circuit methods**: they cause the earlier intermediate methods to be processed only until the short-circuit method can be evaluated.

Stream sources

Streams can be obtained in a number of ways:

- From a **Collection** via the **stream()** and **parallelStream()** methods;
- From an array via **Arrays.stream(Object[])**;
- From static factory methods on the stream classes, such as **Stream.of(Object[])**, **IntStream.range(int, int)** or **Stream.iterate(Object, UnaryOperator)**;
- The lines of a file can be obtained from **BufferedReader.lines()**;
- Streams of file paths can be obtained from methods in **Files**;
- Streams of random numbers can be obtained from **Random.ints()**;
- Generators, like **generate** or **iterate**;
- Several other methods in the JDK...

Intermediate Operations

- An intermediate operation keeps a stream open for further operations. Intermediate operations are **lazy**.
- Several intermediate operations are conceptually higher-order: have arguments of **functional interfaces**, thus **lambdas** can be used

```
interface Stream<T>{...
Stream<T> filter(Predicate<? super T> predicate) // filter

IntStream mapToInt(ToIntFunction<? super T> mapper) // map f:T -> int

<R> Stream<R> map(Function<? super T,? extends R> mapper) // map f:T->R

Stream<T> peek(Consumer<? super T> action) //performs action on elements

Stream<T> distinct() // remove duplicates - stateful

Stream<T> sorted() // sort elements of the stream - stateful

Stream<T> limit(long maxSize) // truncate

Stream<T> skip(long n) // skips first n elements
}
```

Using peek...

- **peek** does not affect the stream
- A typical use is for debugging

```
IntStream.of(1, 2, 3, 4)
    .filter(e -> e > 2)
    .peek(e -> System.out.println("Filtered value: " + e))
    .map(e -> e * e)
    .peek(e -> System.out.println("Mapped value: " + e))
    .sum();
```

- What does it print?

Terminal Operations

- A **terminal operation** is the final operation on a stream. Once a terminal operation is invoked, the stream is consumed and is no longer usable.
- Typical use: collect values in a data structure, reduce to a value, print or other side effects.

```
interface Stream<T>{...
void forEach(Consumer<? super T> action)

Object[] toArray()

T reduce(T identity, BinaryOperator<T> accumulator) // fold

Optional<T> reduce(BinaryOperator<T> accumulator) // fold

Optional<T> min(Comparator<? super T> comparator)

boolean allMatch(Predicate<? super T> predicate) // short-circuiting

boolean anyMatch(Predicate<? super T> predicate) // short-circuiting

Optional<T> findAny() // short-circuiting
}
```

Types of Streams

- Streams only for reference types, int, long and double
 - Minor primitive types are missing

```
"Hello world!".chars()  
    .forEach(System.out::print) ;
```

```
// prints
```

```
721011081081113211911111410810033
```

```
// fixing it:
```

```
"Hello world!".chars()  
    .forEach(x -> System.out.print((char) x)) ;
```

From Reduce to Collect: Mutable Reduction

- Suppose we want to concatenate a stream of strings.
- The following works:

```
String concatenated = listOfStrings  
    .stream()  
    .reduce("", String::concat)
```

- ...but is highly inefficient (it builds one new string for each element)
- Better to “accumulate” the elements in a mutable object (a `StringBuilder`, a collection, ...)
- The ***mutable reduction operation*** is called **`collect()`**. It requires three functions:
 - a **`supplier`** function to construct new instances of the result container,
 - an **`accumulator`** function to incorporate an input element into a result container,
 - a **`combiner function`** to merge the contents of one result container into another.

```
<R> R collect( Supplier<R> supplier,  
               BiConsumer<R, ? super T> accumulator,  
               BiConsumer<R, R> combiner);
```

Mutable reductions: examples

- Collecting the String representations of the elements of a stream into an ArrayList:

```
// no streams
ArrayList<String> strings = new ArrayList<>();
for (T element : stream) {
    strings.add(element.toString());
}
```

```
// with streams and lambdas
ArrayList<String> strings =
    stream.collect(() -> new ArrayList<>(), //Supplier
        (c, e) -> c.add(e.toString()),    // Accumulator
        (c1, c2) -> c1.addAll(c2));       //Combining
```

```
// with streams and method references
ArrayList<String> strings = stream.map(Object::toString)
    .collect(ArrayList::new, ArrayList::add, ArrayList::addAll);
```

Mutable reductions: Collectors

- Method `collect` can also be invoked with a `Collector` argument:

```
<R,A> R collect(Collector<? super T,A,R> collector)
```

- A `Collector` encapsulates the functions used as arguments to `collect(Supplier, BiConsumer, BiConsumer)`, allowing for reuse of collection strategies and composition of collect operations.

```
// The following will accumulate strings into an ArrayList:
```

```
List<String> asList = stringStream.collect(Collectors.toList());
```

```
// The following will classify Person objects by city:
```

```
Map<String, List<Person>> peopleByCity =  
personStream.collect(Collectors.groupingBy(Person::getCity));
```

Infinite Streams

- Streams wrapping collections are finite
- Infinite streams can be generated with:
 - iterate
 - generate

```
static <T> Stream<T> iterate(T seed, UnaryOperator<T> f)
```

```
// Example: summing first 10 elements of an infinite stream
```

```
int sum = Stream.iterate(0, x -> x+1).limit(10).reduce(0, (x, s) -> x+s);
```

```
static <T> Stream<T> generate(Supplier<T> s)
```

```
// Example: printing 10 random numbers
```

```
Stream.generate(Math::random).limit(10).forEach(System.out::println);
```

Parallelism

- Streams facilitate parallel execution
- Stream operations can execute either in serial (default) or in parallel

```
double average = persons //average age of all male
    .parallelStream()     // members in parallel
    .filter(p -> p.getGender() == Person.Sex.MALE)
    .mapToInt(Person::getAge)
    .average()
    .getAsDouble();
```

- The runtime support takes care of using multithreading for parallel execution, in a transparent way
- If operations don't have side-effects, **thread-safety is guaranteed** even if non-thread-safe collections are used (e.g.: ArrayList)

Parallelism (2)

- Concurrent mutable reduction supported for parallel streams
 - Suitable methods of **Collector**
- Order of processing stream elements depends on serial/parallel execution and intermediate operations

```
Integer[] intArray = {1, 2, 3, 4, 5, 6, 7, 8 };
List<Integer> listOfIntegers = new ArrayList<>(Arrays.asList(intArray));
listOfIntegers .stream()
                .forEach(e -> System.out.print(e + " "));
// prints: 1 2 3 4 5 6 7 8
listOfIntegers .parallelStream()
                .forEach(e -> System.out.print(e + " "));
// may print: 3 4 1 6 2 5 7 8
```

SplitIterator: Streams from collections

- A stream wrapping a collection uses a **Splititerator** over the collection
- This is the parallel analogue of an **Iterator**: it describes a (possibly infinite) collection of elements with support for
 - **applying an action to the next element**

```
boolean tryAdvance(Consumer<? Super T> action)
```

- **applying an action to all remaining elements**

```
void forEachRemaining(Consumer <? super T> action)
```

- **splitting** off some portion of the input **into another spliterator** which can be processed in parallel.

```
Spliterator<T> trySplit()
```

- At the lowest level, all streams are driven by a spliterator.

Critical issues

- Non-interference
 - Behavioural parameters (like lambdas) of stream operations should not affect the source (*non-interfering behaviour*)
 - Risk of **ConcurrentModificationExceptions**, even if in single thread
- Stateless behaviours
 - Stateless behaviour for intermediate operations is encouraged, as it facilitates parallelism, and functional style, thus maintenance
- Parallelism and thread safety
 - For parallel streams with side-effects, ensuring thread safety is the programmers' responsibility

Interference: an example

```
try {
    List<String> listOfStrings =
        new ArrayList<>(Arrays.asList("one", "two"));

    String concatenatedString = listOfStrings
        .stream()
        // Don't do this! Interference occurs here.
        .peek(s -> listOfStrings.add("three"))
        .reduce((a, b) -> a + " " + b)
        .get();

    System.out.println("Concatenated string: " + concatenatedString);
} catch (Exception e) {
    System.out.println("Exception caught: " + e.toString());
}
```

MONADS IN JAVA....

Monads in Java: Optional and Stream

```
public static <T> Optional<T> of(T value)
// Returns an Optional with the specified present non-null value.

<U> Optional<U> flatMap(Function<? super T,Optional<U>> mapper)
/* If a value is present, apply the provided Optional-bearing mapping
function to it, return that result, otherwise return an empty
Optional. */
```

```
static <T> Stream<T> of(T t)
// Returns a sequential Stream containing a single element.

<R> Stream<R> flatMap(
    Function<? super T,? extends Stream<? extends R>> mapper)
/* Returns a stream consisting of the results of replacing each element
of this stream with the contents of a mapped stream produced by applying
the provided mapping function to each element. */
```

Functional programming and monads in Java

- About the way monads entered the Java landscape I suggest reading the slides on **Monadic Java** by Mario Fusco.
- More on functional programming in Java in the book **Java 8 in action**

